



Paula Brajczewska, 12121

Łomża, 08.06.2026

Mateusz Drewnowski, 12129

Przedmiot: Inżyniera oprogramowania

Informatyka I stopnia, studia stacjonarne

Semestr IV

System zarządzania schroniskiem dla zwierząt

1. Wprowadzenie

Tematem niniejszego projektu jest zaprojektowanie oraz implementacja zorientowanego obiektowo systemu informatycznego wspomagającego zarządzanie schroniskiem dla zwierząt. Celem aplikacji jest ułatwienie zarządzania schroniskiem, funduszami oraz zwierzętami. Aplikacja ma minimalizować ryzyko błędów ludzkich, zapobieganie przepełnieniu placówki oraz możliwość wyszukiwania zwierząt spełniających kryteria potencjalnych przyszłych właścicieli, lecz przede wszystkim ma na celu automatyzację wielu procesów w zarządzaniu schroniskiem.

2. Wykorzystane narzędzia do projektu

Aplikacja została zaprojektowana przy pomocy poniższych narzędzi

- Język programowania: Java, umożliwiający łatwe przeprowadzenie testów oraz czytelność kodu.
- Środowisko programistyczne: IntelliJ IDEA Community Edition.
- Architektura testów: JUnit 5 - wykorzystany do przeprowadzenia testów.

3. Opis funkcji programu

3.1 Rejestracja nowego zwierzęcia

Metoda *zarejestrujNoweZwierze* odpowiada za rejestrację nowego zwierzęcia w schronisku. Przed utworzeniem obiektu system wykonuje dwa testy: sprawdza warunek przepełnienia schroniska na podstawie wolnych boksów oraz przeszukuje

bazę w celu wykluczenia duplikacji klucza głównego *idZwierzecia*. W przypadku wykrycia anomalii aplikacja przerywa transakcję, rzucając wyjątek.

3.2 Sprawdzanie wolnych boksów

Funkcja *sprawdzWolneBoksy* oblicza w czasie rzeczywistym ilość wolnych boksów schroniska. Jej najważniejszą cechą jest pomijanie w obliczeniach zwierząt, które są oznaczone jako adoptowane. Dzięki temu boksy zajmowane przez adoptowane zwierzęta są automatycznie zwalniane dla nowych podopiecznych.

3.3 Zmiana statusu na adoptowany

Sprawdza, czy zwierzę o odpowiednim ID znajduje się w bazie - w przypadku braku takiego zwierzęcia, wyrzuca błąd oraz blokuje “podwójną adopcję” - nie da się adoptować zwierzęcia, które już jest oznaczone jako adoptowane. Sprawdza również stan obiektu bezpośrednio przed operacją oraz bezpośrednio po operacji.

3.4 Kalkulator kosztów wyżywienia

Narzędzie pozwalające na szacowanie dobowych kosztów wyżywienia schroniska. Funkcja *obliczKosztWyzywienia* pobiera masę ciała zwierzęcia oraz oczekiwany standard wyżywienia, a następnie mapuje stawkę cenową na podstawie wewnętrznego słownika. Wynik kalkulacji jest zaokrąglany do dwóch miejsc po przecinku za pomocą metody, aby odzwierciedlić zapis przy walutach.

3.5 Wyszukiwanie zwierzęcia po rasie i wieku

Funkcja *szukajPoRasieIWieku* jest dedykowana dla osób przyszłych właścicieli. Pozwala na wyszukiwanie dostępnych zwierząt do adopcji z filtrem. Metoda przyjmuje dwa parametry wejściowe: rasę oraz górną granicę wieku i zwraca listę dostępnych pupili do adoptowania.

4. Diagramy UML

Poniżej przedstawiono diagramy UML dla aplikacji:

4.1 Diagram Przypadków Użycia (Use Case Diagram)

Diagram przypadków użycia uwzględnia podział ról w systemie. Posiada on dwóch aktorów: Pracownik Schroniska oraz Potencjalny właściciel (zwierzęcia). Zainteresowany adopcją użytkownik posiada ograniczony dostęp do systemu - jego uprawnienia obejmują wyłącznie przeglądanie zwierząt do adopcji na podstawie rasy oraz wieku. Pełne uprawnienia administracyjne (rejestracja, modyfikacja statusów, zarządzanie finansami) posiada aktor Pracownik Schroniska.

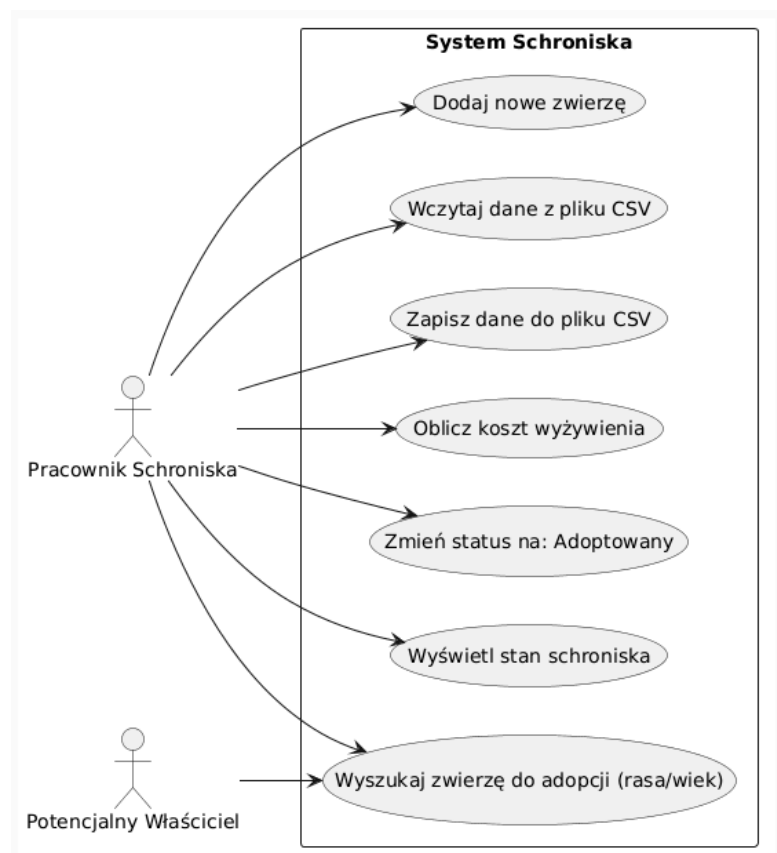


Diagram nr 1 - diagram Use Case

4.2. Diagram Klas (Class Diagram)

Klasy odzwierciedlają logikę obsługi Potencjalnego Właściciela. Klasa **Main** jako interfejs użytkownika (UI) pozwala na wywołanie dedykowanej dla *Potencjalnego Właściciela* metody **szukajPoRasieIWiek** z klasy **Schronisko**. Separacja ta zapewnia dodatkową warstwę bezpieczeństwa, dzięki której użytkownik

nie ma dostępu do niepożądanych operacji, które są zarezerwowane dla Pracownika Schroniska.

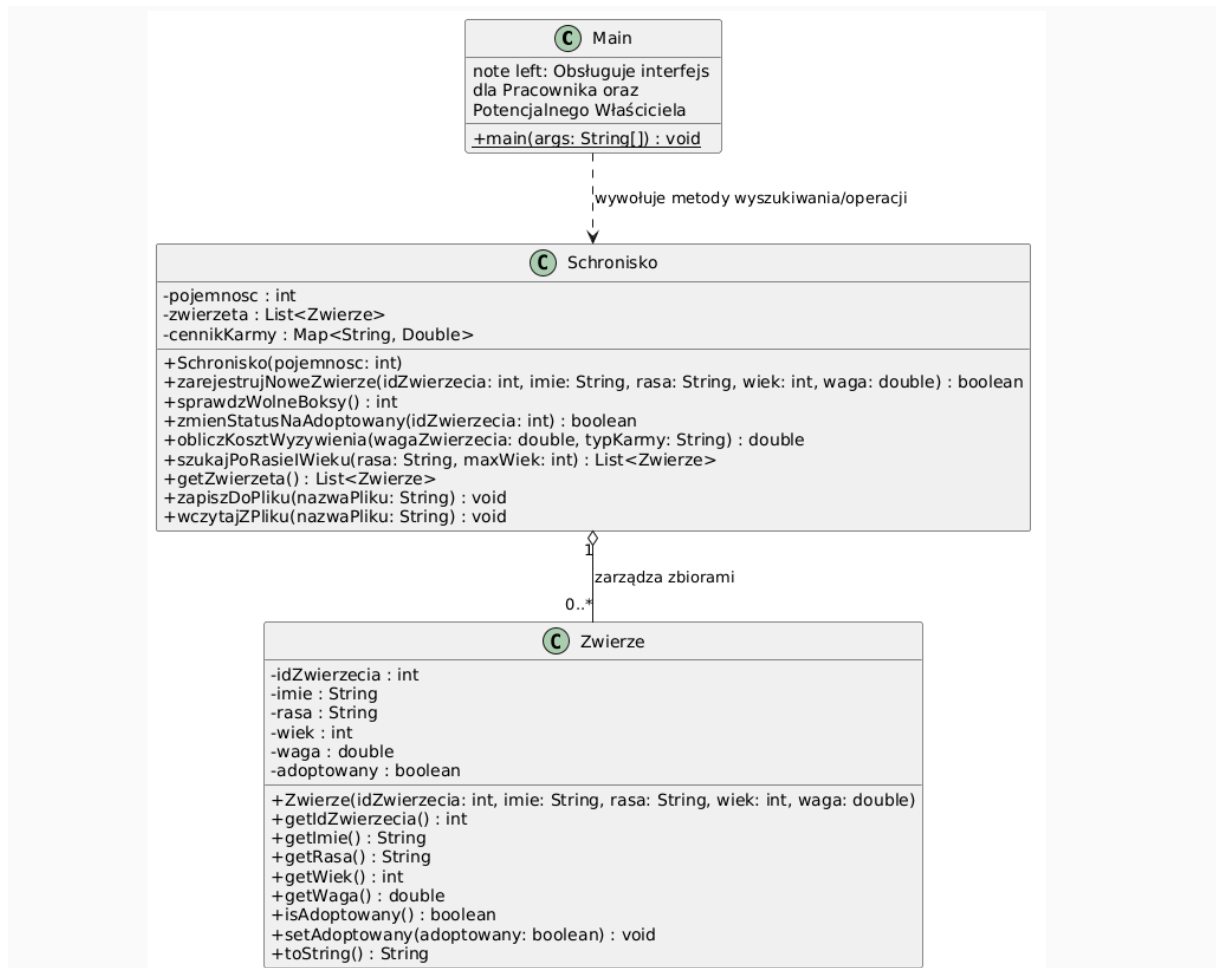


Diagram nr 2 - diagram klas

4.3. Diagram Czynności (Activity Diagram)

Diagram czynności obrazuje, w jaki sposób działa algorytm filtrowania danych uruchamiany w momencie, gdy Potencjalny Właściciel szuka zwierzaka. System w pętli sprawdza trzy warunki logiczne: zgodność rasy, wiek poniżej lub równy kryterium oraz to, czy zwierzę jest dostępne do adopcji. Dzięki temu użytkownik widzi wyłącznie zwierzęta aktualnie przebywające w schronisku.

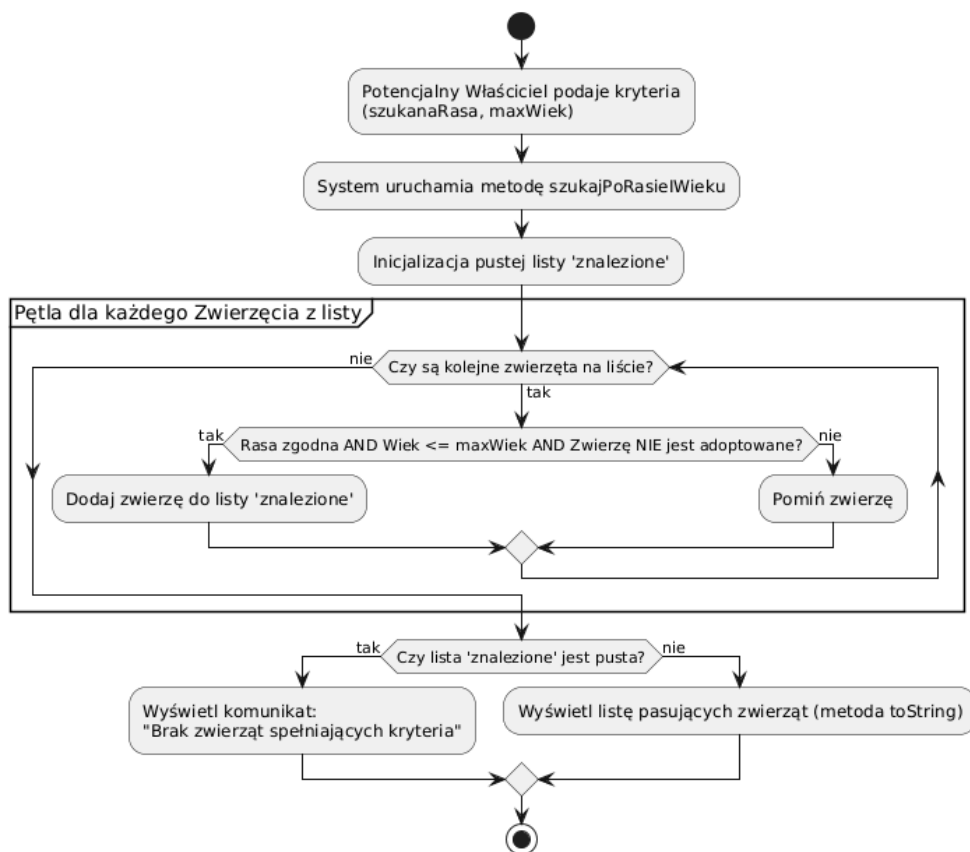


Diagram nr 3 - diagram aktywności

4.4. Diagram Sekwencji (Sequence Diagram)

Diagram sekwencji przedstawia przebieg komunikacji w trakcie wyszukiwania. Aktor Potencjalny Właściciel wyszukuje zwierzę. Obiekt klasy Schronisko przetwarza zapytanie, odpytując poszczególne obiekty klasy Zwierze o ich parametry, a na koniec przefiltrowana lista jest zwracana i prezentowana użytkownikowi.

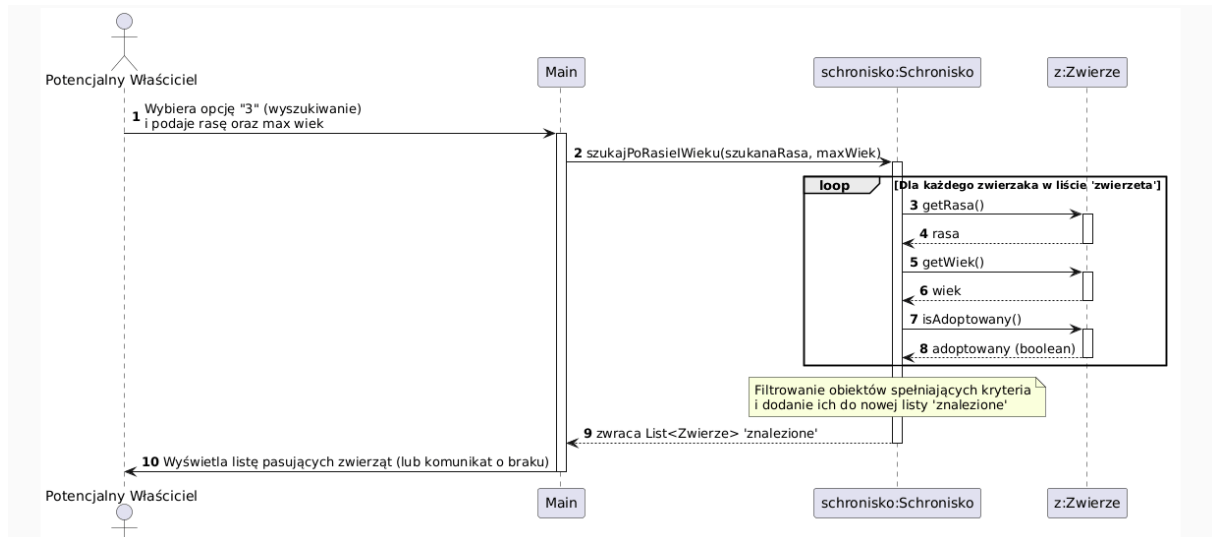


Diagram nr 4 - diagram sekwencji

5. Kod programu

Poniżej zamieszczono link do Githuba z kodem programu:

<https://github.com/MattWoodz/Schronisko-Przylulisko-Projekt#>

6. Przeprowadzone testy

Funkcja 1: zarejestrujNoweZwierze

testZarejestrujNoweZwierzeSukces - sprawdza ścieżkę pozytywną, czyli pomyślne dodanie zwierzęcia i powiększenie rozmiaru listy bazy danych.

testZarejestrujNoweZwierzeBrakMiejsca - weryfikuje zabezpieczenie systemu w sytuacji, gdy użytkownik próbuje dodać zwierzę, przekraczając limit pojemności schroniska.

testZarejestrujNoweZwierzeDuplikatId - sprawdza, czy system poprawnie blokuje próbę rejestracji zwierzęcia z numerem ID, który już istnieje w bazie.

testZarejestrujNoweZwierzeWielePoprawnych - testuje wydajność i integralność bazy przy sekwencyjnym dodawaniu wielu poprawnych obiektów.

Funkcja 2: sprawdzWolneBoksy

testSprawdzWolneBoksyPusteSchronisko - potwierdza, że dla nowo utworzonego schroniska funkcja zwraca maksymalną pojemność.

testSprawdzWolneBoksyPoDodaniu - sprawdza matematyczną poprawność spadku liczby miejsc dokładnie o jeden po udanej rejestracji.

testSprawdzWolneBoksyPoAdopcji - weryfikuje, czy system poprawnie zwalnia boks i powiększa pulę miejsc po tym, jak zwierzę zmieni status na zaadoptowane.

testSprawdzWolneBoksyPelne - sprawdza zachowanie funkcji w skrajnym przypadku całkowitego zapełnienia placówki (oczekiwany wynik to 0).

Funkcja 3: zmienStatusNaAdoptowany

testZmienStatusSukces - weryfikuje bezproblemowe przejście procesu adopcji dla prawidłowych danych.

testZmienStatusNieistniejaceId - sprawdza odpowiedź systemu (IllegalArgumentException) w przypadku wpisania błędnego lub nieistniejącego identyfikatora zwierzęcia.

testZmienStatusJuzAdoptowane - weryfikuje logikę zabezpieczającą przed ponowną próbą wydania do adopcji zwierzęcia, które znalazło już dom (IllegalStateException).

testZmienStatusSprawdzenieWlasciwosci - bada bezpośrednio hermetyzację obiektu, upewniając się, że wewnętrzna flaga logiczna zmienia się z false na true.

Funkcja 4: obliczKosztWyzywienia

testObliczKosztStandard - sprawdza poprawność matematyczną algorytmu dla standardowego mnożnika karmy.

testObliczKosztPremium - weryfikuje poprawne wyliczenia dla droższego, zdefiniowanego w słowniku wariantu karmy.

testObliczKosztUjemnaWaga - bada odporność algorytmu na wprowadzanie danych nielogicznych, wymuszając przechwycenie ujemnej wartości wagowej i wyrzucenie błędu.

testObliczKosztBlednyTypKarmy - sprawdza system walidacji tekstowej w przypadku podania nazwy karmy, która nie istnieje w cenniku.

Funkcja 5: szukajPoRasieIWieku

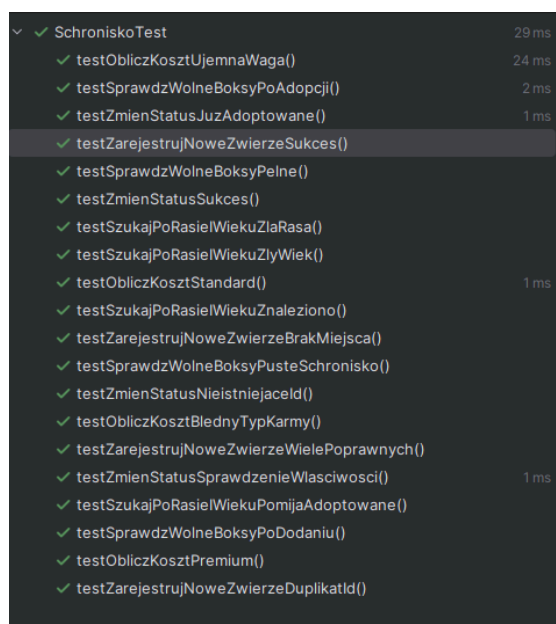
testSzukajPoRasieIWiekuZnaleziono - potwierdza poprawne wyodrębnienie i zwrócenie zwierzęcia idealnie dopasowanego do kryteriów.

testSzukajPoRasieIWiekuZlyWiek - sprawdza mechanizm wykluczania, upewniając się, że zwierzęta starsze niż zadany limit nie pojawiają się w wynikach.

testSzukajPoRasieIWiekuZlaRasa - weryfikuje mechanizm wykluczania ze względu na niezgodność ciągu znaków (inny tekst rasy).

testSzukajPoRasieIWiekuPomijaAdoptowane - potwierdza poprawne działanie logiki biznesowej, ukrywając w wyszukiwarce zwierzęta, które nie są już dostępne fizycznie w schronisku.

Poprawność działania wszystkich wymienionych wyżej funkcji biznesowych została potwierdzona za pomocą frameworka testowego. Poniższy zrzut ekranu prezentuje pomyślne wykonanie wszystkich 20 przypadków testowych w środowisku programistycznym.



Zrzut ekranu nr 1 - Wykonanie wszystkich testów zakończone pomyślnie.

7. Podział obowiązków w grupie:

Paula Brajczewska:

- przygotowanie sprawozdania,
- przygotowanie diagramów UML,
- zaprojektowanie oraz implementacja funkcji `szukajPoRasieIWieku`,
wprowadzenie nowego aktora `Uzytkownik`
- implementacja funkcji `obliczKosztWyzywienia`,
- wprowadzanie poprawek w kodzie programu oraz dokumentacji
- zadbanie o estetykę i poprawność dokumentacji

Mateusz Drewnowski:

- stworzenie i przeprowadzenie testów jednostkowych
- opisanie przeprowadzonych przypadków testowych w dokumentacji
- implementacja funkcji `zarejestrujNoweZwierze`, `sprawdzWolneBoksy`,
`zmienStatusNaAdoptowany`
- zaprojektowanie głównego menu konsolowego
- skompletowanie kodu, wprowadzanie poprawek i usprawnień